

## An Agile process model for product derivation in software product line engineering

Pádraig O’Leary<sup>1,\*</sup>, Fergal McCaffery<sup>1,2</sup>, Steffen Thiel<sup>3</sup> and Ita Richardson<sup>1</sup>

<sup>1</sup>*Lero, The Irish Software Engineering Research Centre, University of Limerick, Ireland*

<sup>2</sup>*Dundalk Institute of Technology, Dundalk, Ireland*

<sup>3</sup>*Department of Computer Science, Furtwangen University of Applied Sciences, Germany*

### SUMMARY

Software product lines (SPL) and Agile practices have emerged as new paradigms for developing software. Both approaches share common goals; such as improving productivity, reducing time to market, decreasing development costs and increasing customer satisfaction. These common goals provide the motivation for this research. We believe that integrating Agile practices into SPL can bring a balance between agility and formalism. However, there has been a little research on such integration. We have been researching the potential of integrating Agile approaches in one of the key SPL process areas, product derivation (PD). In this paper, we present an outline of our Agile process model for PD that was developed through industry-based case study research. Copyright © 2010 John Wiley & Sons, Ltd.

Received 14 May 2010; Accepted 14 May 2010

KEY WORDS: software product lines; product derivation; Agile approaches

### 1. INTRODUCTION

Both Agile and software product lines (SPL) development paradigms are being promoted as means of reducing time to market, increasing productivity, and gaining cost effectiveness and efficiency of software development efforts [1]. Furthermore, both approaches assume that requirement changes will occur and can be managed effectively [1]. These goals (shared by Agile and SPL) open the possibilities of introducing Agile practices into SPL activities. There are, however, several challenges involved in integrating Agile approaches in SPL development due to certain differences that exist in the philosophies of both approaches, such as design and change management strategies [1, 2]. Moreover, Agile approaches do not purport to develop flexible artefacts for reuse [2, 3] or develop and maintain rigorous and extensive documentation as required by SPL [3].

Our research in SPL is aimed at improving the product derivation (PD) process, which purports to develop new products by utilizing core assets of an SPL such as feature models, architecture models, and code artefacts [4], through the adoption of Agile practices.

In this paper, we present our research results on the development of an Agile process model for PD (A-Pro-PD). A-Pro-PD was developed as a part of Process reference model for PD (Pro-PD). Pro-PD was developed at Lero (the Irish Software Engineering Research Centre) with the goal of defining a generic process reference model for PD. Pro-PD can be used as a foundation for

\*Correspondence to: Pádraig O’Leary, Lero, The Irish Software Engineering Research Centre, University of Limerick, Ireland.

†E-mail: padraig.oleary@lero.ie

situation-specific process approaches to PD. Pro-PD, its development, and its validation are also described in [5]. As part of the development of Pro-PD, we adapted it through Use-by-Specialization [6] to incorporate Agile principles. This adapted version is called A-Pro-PD.

We decided to concentrate on PD as it is considered one of the most important and challenging SPL 'activities' [7], and the activity that has the most to gain from the successful implementation of Agile practices. We believe that any successful effort to introduce Agile practices in the PD process can make SPL significantly more effective and efficient. While some research in the area of Agile SPL has been reported [1–3, 8–10], much of this has been related to identifying the opportunities and challenges. There has been a little research conducted on the use of Agile approaches in the PD process, whereas no Agile approach for PD have been developed. This is a first attempt at bridging this gap.

A-Pro-PD is a lightweight approach to PD, minimising the amount of up-front investment required and therefore making SPL more accessible to small organizations with limited resources. For larger organizations, A-Pro-PD could bring a balance between formalism and agility, helping individual product teams deliver products with the best possible quality. A combination of Agile and SPL is expected to create a leaner but more disciplined PD process [8].

The remainder of this paper is organized as follows: Section 2 describes the key concepts of SPL and Agile practices. Section 3 discusses the research methodology. Section 4 presents an overview of A-Pro-PD. In Section 5, we discuss in detail the Agile aspects of the A-Pro-PD. The paper concludes in Section 6 with a summary and an outlook of future work.

## 2. BACKGROUND AND MOTIVATION

In the following section, we discuss the main concepts of Agile and SPL that underpin our proposal for integrating the two.

### 2.1. Software product lines

An SPL is a set of software-intensive systems that share a common, managed set of features satisfying the specific needs of a particular market segment or mission and that are developed from a common set of core assets in a prescribed way [8]. The SPL approach makes a distinction between domain engineering, where a common platform for an arbitrary number of products is designed and implemented, and application engineering, where a product is derived based on the platform components [10]. It is during application engineering that the individual products within a product line are constructed. The process of creating these individual products using the shared artefacts is known as the PD process [4].

The underlying assumption of PD is that 'the investments required for building the reusable assets during domain engineering are outweighed by the benefits of rapid derivation of individual products' [4]. This assumption might not hold if inefficient derivation practices diminish the expected gains.

A number of publications discuss the difficulties associated with PD [4, 11–13]. Hotz *et al.* [13] describe the process as 'slow and error prone even if no new development is involved'. Deelstra *et al.* [4] observe that the derivation of individual products from shared software assets is still a time-consuming and expensive activity in many organizations. The authors state that 'there is a lack of methodological support for application engineering and, consequently, organizations fail to exploit the full benefits of software product families'. 'Guidance and support are needed to increase efficiency and to deal with the complexity of PD' [12].

### 2.2. Agile practices

Agile methods have recently gained popularity among large numbers of companies as a mechanism for reducing costs and increasing ability to handle change in dynamic market conditions. In the 2000s, interest in Agile methods increased dramatically [14]. Agile methods have been adopted in different types of software projects and in various application domains [15].

Researchers have shown that the use of Agile methods can assist in product manageability, visibility, and team communication [16] as well as ensure frequent feedback from the customer [17]. Researchers and practitioners have proposed several software development approaches based on the principles of the Agile manifesto [18]. The Agile manifesto defines four basic core values:

- Individuals and interactions over processes and tools;
- Working software over comprehensive documentation;
- Customer collaboration over contract negotiation;
- Responding to change over following a plan.

In addition to these four values, the Agile manifesto defines a set of principles that form guidelines for development. The Agile manifesto provides a foundation for all of the Agile methods including: eXtreme Programming (XP) [19] and Scrum [20].

XP evolved from the problems that were caused by the long development cycles of traditional development models [21].

The individual practices of XP are not new; however, those practices have been collected and lined up to function with each other in a novel way. The term 'extreme' comes from taking these commonsense principles and practices to extreme levels [22]. In some instances, selected sets of XP practices have even been used in the field of safety critical systems [23]. In such cases, the use of XP practices has reportedly improved the quality by 53% compared with the plan-driven software development project.

Scrum provides a project management framework that focuses development into 30-day Sprint cycles in which a specified set of Backlog features are delivered [20]. The core practice in Scrum is the use of daily 15-min team meetings for coordination and integration. Scrum does not define any specific software development techniques. Scrum concentrates on how team members should function in order to produce good quality code and maintain flexibility in a changing environment.

Although XP and Scrum are based on a common guideline defined by the Agile manifesto, they vary in focus and presentation. XP emphasizes the technical elements of the development lifecycle, whereas Scrum concentrates on the project management. One of the central principles of the Agile approach is the focus on people. People, coupled with effectiveness and manoeuvrability, are considered the primary drivers of a project's success [24].

### *2.3. Agile practices and SPL*

Recently there has been a growing interest in exploring the possibilities of integrating Agile approaches into the SPL development process. In conjunction with the 2006 Software Product Line Conference (SPLC), a new workshop was arranged called Agile Product Line Engineering (APLE). From the workshop it was recognized that both Agile approaches and SPL share several common goals [1, 8, 10] such as:

- Increasing the productivity of teams;
- Reducing the product's time to market;
- Reducing the development costs;
- Improving customer satisfaction.

Both approaches assume that requirement changes during the development will occur and Agile-based testing could benefit SPL development [1]. Furthermore, both development paradigms are being promoted as a means of reducing time to market, increasing productivity, and gaining cost effectiveness and efficiency of software development efforts [1]. These goals (shared by Agile and SPL) open the possibilities of introducing Agile practices into SPL activities.

While a motivation for combining Agile and SPL approaches can be found, there is also recognition of the potential challenges of any integration effort due to certain differences that exist in the philosophies of both approaches such as design and change management strategies [1, 2].

Both approaches have different design strategies. Agile focuses on a simple design, whereas SPL has an emphasis on rigorously designing and systematically maintaining SPL architectures [2].

Agile approaches do not develop flexible artefacts for reuse [2, 3]. SPL tends toward long-lived life cycles, which means that the maintaining of software is necessary [3]. SPL targets at satisfying the needs of several customers rather than satisfying individual needs [3]. SPL and Agile approaches include different strategies for change management: in Agile, it is incremental development and in SPL the focus is on platform artefacts [1]. SPL architecture needs to be flexible to handle requirements of several customers [25], Agile does not.

Despite having similar goals, both Agile and SPL have different mechanisms and logistics for achieving their respective goals. SPL aims to fulfil the requirements of several customers at the same time rather than concentrating on meeting the individual needs of a particular customer [2]. Customer involvement is much more intensive in Agile approaches where developers work closely with customers on a daily basis. Agile and SPL paradigms also differ in terms of role and importance of software design. Agile approaches do not emphasize the importance of rigorous design and documentation; rather Agile approaches advocate implementing required functionality and then documenting the prepared code by reconstruction and refactoring. On the other hand, design and documentation are very vital activities in SPL as platform assets need to be appropriately documented to support their reusability [1].

As a result of these challenges, there are a number of risks associated with an Agile SPL approach. If an SPL-based architecture is tailored to be more Agile, there is a danger that valuable architecture that supports other products in the family may be damaged [1]. Traceability management as well as maintaining of components in Agile approaches can be difficult without explicit knowledge [10] and no tool support for Agile SPL approach exists [2].

Despite the documented challenges and risks mentioned, there are some suggested approaches for integrating Agile practices in SPL. Some suggestions include that the Planning Game [3] from the XP methodology can increase the agility in SPL organizations, particularly for gathering and negotiating the product requirements. The use of Agile practices during application engineering activities is suggested [2], in particular to perform product tailoring [9] and to support the collaborative working of stakeholders [1].

An Agile SPL approach may have the potential of supporting the effective and efficient development of much larger families of products than a traditional SPL approach. Moreover, the Agile project management approach Scrum can be used to manage the PD process and development practices like XP or test-driven development can be used for developing platform and/or product components.

### 3. RESEARCH APPROACH

A-Pro-PD was developed, as part of Pro-PD, with the goal of defining a generic Agile process model for PD. The preparatory stage of this research was conducted as an extensive literature review. The research aimed to identify the fundamental practices of PD and Agile approaches. The initial results were further developed and assessed through a series of iterative workshops over a four-month period. Evidence and feedback from SPL and Agile experts were collected from these organized workshops. The output of this four-month iterative development stage was version one of Pro-PD [26].

We extended version one through case study research with Robert Bosch GmbH<sup>‡</sup>. Here we investigated PD practices within an automotive systems sub-unit. The systems produced consist of both hardware (such as processors, sensors, connectors, and housing) and software. Data collection involved studying internal company documentation, an onsite visit to their headquarters, and a two-day workshop with key employees. Based on the knowledge garnered on the derivation practices within the company, we identified the areas with potential for the integration of Agile methods. The output of this research was a technical report [27], where we documented our recommendations on

<sup>‡</sup><http://www.bosch.com>.

the use of Agile practices within Bosch automotive business units. By generalising and discussing our observations, we revised version one and developed version two of Pro-PD [28].

The research was further developed through two research collaborations. The first was a six month visit to LASSY<sup>§</sup>, where Pro-PD and FIDJI [29] were mapped. The second was a collaboration project with Doppler Laboratory where we investigated the application of their DOPLER<sup>UCon</sup> [12] tool within the Pro-PD. The output of this phase of the research was [30].

We performed further validation of Pro-PD through an inter-model evaluation with the SEI Product Line Practice Framework [31].

We used the Eclipse Process Framework (EPF) [The Eclipse Foundation, [32]] to model A-Pro-PD. By enabling inbuilt process variability within EPF we can select, tailor, or remove content from our process in order to strike the right balance for a particular situation. Moreover, a documented process model is a good starting point for the integration of non-standard techniques such as Agile practices, at appropriate times in the development process [26, 33, 34].

#### 4. A-Pro-PD

A-Pro-PD is intended to offer process guidance to small, co-located teams and support the adoption of Agile practices in PD. The instantiation of A-Pro-PD is intended to:

- Encourage the use of iterative development cycles in PD.
- Minimize the risk associated with ‘big bang’ releases where large products are assembled just before product delivery.
- Increase customer involvement in the derivation process.

PD approaches in the literature [27] and industry practice observed through this research (c.f. Section 3) follow a phased structure. These phases are broadly speaking requirements analysis, product configuration and artefact reuse, and finally product specific development and testing. These phases are reflected in the structure of the A-Pro-PD. Through our research into Agile methods, we have applied iterative and incremental approaches within this phased lifecycle.

There are two layers to A-Pro-PD (see Figure 1). These are the phase increments layer and iteration lifecycle layer. Phase increments are short units of work on a particular aspect of the derivation process e.g., configuring platform components. The iterative lifecycle layer structures these phase increments to deliver stable builds of the product that incrementally progress towards the iteration objectives. These iterations result in regular product releases.

The three principal phases, consisting of essential activities required during any PD project, within the A-Pro-PD are:

- Preparing for Derivation;
- Product Configuration;
- Product Development and Testing.

##### 4.1. *Preparing for Derivation*

*Preparing for Derivation* phase determines the product requirements and performs the project management tasks for a specific iteration. The phase forms the product-specific requirements based on customer requirements and negotiation with the platform team. Requirements are prioritized and assigned to development iterations. The purpose of this phase is to achieve agreement among all the stakeholders on the *Product Requirements*. On subsequent iterations, this phase is used to plan and manage product iterations. The main objectives of this phase are as follows:

- Define the product requirements;
- Scope the Product;

<sup>§</sup>Laboratory of Advanced Software Systems (LASSY), University of Luxembourg.

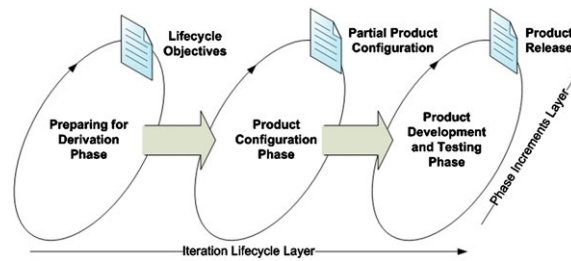


Figure 1. Agile process model for product derivation.

- Define guidance for decision makers;
- Allocate requirements for implementation in a specific iteration;
- Plan the project;
- Identify role and task structures.

#### 4.2. Product Configuration

The goal of the *Product Configuration* phase is to create a partial product configuration that maximizes the benefits of the platform artefacts and minimizes the amount of product-specific development required. The partial product configuration is derived based on the product-specific requirements and by using the available assets. Theoretically at this stage, the partial product configuration could satisfy the product requirements for a specific iteration and testing should begin. However, this is the ideal case and assumes that all the product requirements are covered by the platform. In most cases, some additional development will be required. This additional development occurs in the next phase.

The main objectives of this phase are as follows:

- Create the Product Build;
- Make maximum use of the platform artefacts.

#### 4.3. Product development and testing

This phase focuses on satisfying requirements that could not be satisfied through the reuse of platform assets through product-specific development and transitioning the software into the customers environment. Transitioning the software involves achieving customer agreement that the product is complete. The purpose of this phase is to ensure that the product is ready for delivery to users.

The number of iterations in the phase varies from one iteration (for a simple system requiring primarily minor bug fixing) to many iterations (for a complex system involving adding features and performing activities) to make the business transition from using the old system to using the new system.

When the *Product Requirements* have been met, the project is in a position to be closed. For some products, the end of the current project lifecycle may coincide with the beginning of the next lifecycle, leading to the next generation of the same product. The main objectives of this phase are as follows:

- Test the product to ensure that it meets the product requirements;
- Correct defects;
- Modify the software if unforeseen problems arise;
- Provide feedback to platform team on the usage of platform.

#### 4.4. Phase milestones

Phase milestones are used as gateways for phase endings. If a milestone is reached, the project is ready to move to the next phase. Each iteration of the project provides an increment in functionality or design.

Each phase has a specific focus and objective. At the end of the *Preparing for Derivation* phase is the first milestone, the *Product Requirements Milestone*. At this point, the product requirements have been defined and allocated for the iteration.

At the end of the Product Configuration phase is the *Product Build Milestone*. At this point, a *Product Build* is constructed.

At the end of the *Product Development and Testing* Phase is the *Product Release Milestone*. At this point, a decision is made on whether the Product Requirements allocated to this iteration have been met.

## 5. INCREASING AGILE IN PD

In this section, we discuss the following Agile elements of the A-Pro-PD:

- Adoption of early and continuous delivery strategy;
- Automation of PD;
- PD iterations;
- Agile testing techniques.

We describe how these elements were identified and the benefits that they can bring to PD.

### 5.1. Adoption of early and continuous delivery strategy

Typically, implementing product-specific features can be time consuming. First, product construction can be substantially delayed due to the Change Control Board (CCB). The CCB scopes new development to gauge the reusability of a requested feature within the product line. Second, development is further delayed if the Product Team defers implementing a feature until the platform team implements the requested platform changes at the product level.

In the A-Pro-PD, we adopt the Agile principle of ‘early and continuous delivery of valuable software’. The product team implements changes at product level. The Platform Team subsequently mines any changes from the product if there is reuse potential.

In Bosch we observed this Agile principle in action. To facilitate early and continuous delivery of software, the product team would not wait for scoping decisions from the CCB. Rather, the product team would negotiate a new platform interface containing required extensions to facilitate new product components before proceeding to develop in parallel against the platform team. When the platform extensions had been implemented and the new platform was released, the product team would check for compatibility issues with the newly developed components.

We recommended [8], the adoption of the Agile practice of pair programming for customer-specific components. Pair programming is suitable for implementing and reviewing any changes at the product level [8]. This helps to produce better quality product code and consequently, improved code for any features that are mined for the platform.

### 5.2. Automation of PD

Automated support for PD is a necessity for managing the complexity and variability inherent in SPLs and according to Kurmann [18], automation is the most important aspect of an Agile SPL. Automated development approaches facilitate the Agile Principle ‘*Welcome changing requirements, even late in development. Agile processes harness change for the customer’s competitive advantage.*’ [12], as automated development techniques allow product teams to implement changing customer requirements late in the development lifecycle and automation enables these changes to be implemented quickly.

However, current process models and tools for automation do not integrate well. All the stakeholders involved in PD are supported in their tasks by different approaches and different automation tools. Because of the difficulty of integrating these different approaches and tools, PD can quickly become an error-prone and tedious task.

In our research collaboration with Dopler Laboratory (cf. Section 3), we investigated how DOPLER<sup>UCOn</sup> [2] tool could be used within the A-Pro-PD. We were particularly interested in its

ability to facilitate Agile approaches. For instance, we observed that although the DOPLER<sup>UCon</sup> tool does not directly support *iterative development* cycles by defining additional attributes for requirements, it could be used to allocate specific requirements to specific iterations.

### 5.3. PD iterations

The identification of PD iterations is a key aspect of deriving high quality, customer satisfying products. According to Carbon *et al.* [2] when adopting an SPL approach, an organization is capable of producing a first version of a product for a specific customer, including the core functionality, quicker than other software development methods. Because of the approved quality of the reusable assets, the customer can get a high-quality product that can be used and evaluated to give feedback. In further iterations, new functionality can be added to the scope of the product line or product specific features can be implemented [27].

In a technical report to Bosch [35], we recommended that they could benefit from applying the planning game practice from the XP methodology for the management of their product iterations during the *Preparing for Derivation* phase. This would assist them in gathering and negotiating product-specific requirements. During customer negotiation, requirements are prioritized and allocated to specific iterations based on priority.

### 5.4. Agile testing techniques

Agile methods propose that testing is carried out frequently, as this helps Agile developers keep their code as error free as possible. We have adopted a phased testing approach in the A-Pro-PD. Based on the principles of integration testing suggested by Muccini [13], the structure and nature of the elements in a product line are leveraged. First, integrate the partial configuration and use a traditional approach to integration testing. Then, based on the observation that at least the partial product configuration works properly, we can incorporate the other product elements. Product construction continues in a phased assembly test approach. For systems testing of partial or fully assembled products traditional system testing techniques can be utilized as no SPL specific methods exist.

## 6. CONCLUSION AND FUTURE WORK

In this paper, the results of research into the adoption of Agile practices in PD were presented. The research into Agile practices in PD is motivated by the fact that despite the widespread adoption of SPL within the industry, PD remains an expensive and error-prone activity [36]. The adoption of Agile practices could improve the PD process. A-Pro-PD provides a means of supporting this adoption.

The development of the Agile approach to PD is a response to calls from the industry for research into this area [8]. The integrated Agile process model could solve many of the problems associated with PD's complex and cumbersome nature.

A-Pro-PD is a lightweight approach to PD, minimising the amount of up-front investment required making SPL more accessible to small organizations with limited resources. The A-Pro-PD may benefit larger organizations by bringing a balance between formalism and agility, helping individual product teams to deliver products with the best possible quality. A combination of Agile and SPL is expected to create a leaner but more disciplined PD process.

Our future work includes an ongoing investigation into the benefits of combining Agile and SPL approaches and the validation of A-Pro-PD, particularly with respect to the expected return on investment. Another area of future work is the combination of Lean Software Development [37] and SPL. As Lean Software Development is based on production methods from product lines at Toyota<sup>‡</sup>, the integration of lean principles in SPL could have interesting results.

<sup>‡</sup><http://www.toyota.com/>.

## ACKNOWLEDGEMENTS

This work is partially funded by IRCSET under grant no. RS/06/167 and by Science Foundation Ireland under grant nos. 03/CE2/I303\_1 and 07/SK/I1299.

## REFERENCES

1. Tian K, Cooper K. Agile and software product line methods: Are they so different? *First International Workshop on Agile Product Line Engineering*, 2006.
2. Carbon R, Lindvall M, Muthig D, Costa P. Integrating product line engineering and agile methods: Flexible design up-front vs. incremental design. *First International Workshop on Agile Product Line Engineering (APLE'06)*, 2006.
3. Navarrete F, Botella P, Xavier F. An approach to reconcile the agile and CMMI context in product line development. *First International Workshop on Agile Product Line Engineering (APLE'06)*, 2006.
4. Deelstra S, Sinnema M, Bosch J. Product derivation in software product families: A case study. *Journal of Systems and Software* 2005; **74**(2):173–194.
5. O'Leary P. *Towards a Product Derivation Process Reference Model for Software Product Line Organisations*. Department of Computer Science and Information Systems: University of Limerick, Limerick, 2009; 241.
6. Braun R, Esswein W, Gehlert A, Weller J. Configuration management for reference models. *Reference Modeling for Business Systems Analysis*, Fettke P, Loos P (eds.). IGI: Hershey PA, U.S.A., 2006; 331–357.
7. Griss ML. Implementing product-line features with component reuse. *ICSR-6 Proceedings of the Sixth International Conference on Software Reuse*. Springer: London, U.K., 2000; 137–152.
8. Kurmann R. Agile SPL-SCM agile software product line configuration and release management. *First International Workshop on Agile Product Line Engineering (APLE'06)*, 2006.
9. Noor M, Rabiser R, Grünbacher P. A collaborative approach for reengineering-based product line scoping. *First International Workshop on Agile Product Line Engineering (APLE'06)*, 2006.
10. Trinidad P, Benavides D, Ruiz-Cortes A, Segura S. Explanations for agile feature models. *First International Workshop on Agile Product Line Engineering (APLE'06)*, 2006.
11. O'Leary P, Richardson I, McCaffery F, Thiel S. Preparing for product derivation: Activities and issues. *Fourth International Conference on Software and Data Technologies—ICSOF 2009*. INSTICC—Institute for Systems and Technologies of Information, Control and Communication: Sofia, Bulgaria, 2009.
12. Rabiser R, Grünbacher P, Dhungana D. Supporting product derivation by adapting and augmenting variability models. *Eleventh International Software Product Line Conference*, 2007.
13. Hotz L, Gunter A, Krebs T. A knowledge-based product derivation process and some ideas how to integrate product development. *Proceedings of Software Variability Management Workshop*, 2003.
14. Mikael L, Muthig D, Dagnino A, Wallin C, Stupperich M, Kiefer D, May J, Kahkonen T. Agile software development in large organizations. *Computing Practices* 2004; **37**(12):38–46.
15. Karlström D. Introducing extreme programming—An experience report. *Third International Conference on eXtreme Programming and Agile Processes in Software Engineering (XP2002)*, 2002.
16. Larman C. *Agile and Iterative Development*. Addison-Wesley Professional: Boston, 2003; 342.
17. Rising L, Janoff NS. The scrum software development process for small teams. *IEEE Software* 2000; **17**(4): 26–32.
18. Beck K, Beedle M, Bennekum Av, Cockburn A, Cunningham W, Fowler M, Grenning J, Highsmith J, Hunt A, Jeffries R, Kern J, Marick B, Martin RC, Mellor S, Schwaber K, Sutherland J, Thomas D. *Manifesto for Agile Software Development*, 2001; 1/3/2006. Available from: <http://agilemanifesto.org/> [10/09/2008].
19. Beck K, Andres C. *Extreme Programming Explained: Embrace Change* (2nd edn). Addison-Wesley Professional: Reading MA, 2004.
20. Schwaber K, Beedle M. *Agile Software Development with Scrum*. Prentice-Hall PTR: Englewood Cliffs NJ, 2001.
21. Beck K. Embracing change with extreme programming. *IEEE Computer* 1999; **32**(10):70–77.
22. Beck K. *Extreme Programming Explained: Embrace Change*. Addison-Wesley Longman Publishing Co Inc.: Boston MA, U.S.A., 1999; 190.
23. Grenning J. *Using XP in a Big Process Company: A Report From the Field*, XP Agile Universe, Raleigh NC, 2001.
24. Highsmith J, Cockburn A. Agile software development: The business of innovation. *IEEE Computer* 2001; **34**(9):120–122.
25. Clements P, Northrop L. *Software Product Lines: Practices and Patterns*. Addison-Wesley Longman Publishing Co., Inc.: Boston MA, U.S.A., 2001.
26. O'Leary P, Ali Babar M, Thiel S, Richardson I. Product derivation process and agile approaches: Exploring the integration potential. *Proceedings of Second IFIP Central and East European Conference on Software Engineering Techniques*. Wydawnictwo NAKOM: Poznań, Poland, 2007.
27. O'Leary P, Thiel S, Botterweck G, Richardson I. *Bosch Technical Report, LERO—The Irish Software Engineering Research Centre*, 2008.

28. O'Leary P, Thiel S, Botterweck G, Richardson I. Towards a product derivation process framework. *Third IFIP TC2 Central and East European Conference on Software Engineering Techniques CEE-SET 2008*, 2008.
29. Perrouin G, Klein J, Guelfi N, Jezequel JM. Reconciling automation and flexibility in product derivation. *Twelfth International Software Product Line Conference (SPLC)*, 2008.
30. O'Leary P, Rabiser R, Richardson I, Thiel S. Important issues and key activities in product derivation: Experiences from two independent research projects. *Software Product Line Conference, Proceedings of the 13th International Software Product Line Conference (SPLC 2009)*, 2009.
31. Northrop L, Clements P. *A Framework for Software Product Line Practice*, version 5.0, 2007; 30th April 2008. Available from: <http://www.sei.cmu.edu/productlines/framework.html> [cited 12/09/2008].
32. *Eclipse Process Framework Composer (EPF)*. IBM, 2007.
33. O'Leary P, Ali Babar M, Thiel S, Richardson I. Towards agile product derivation in software product line engineering. *Fourth International Workshop on Rapid Integration of Software Engineering techniques (RISE)*, 2007.
34. O'Leary P, McCaffery F, Richardson I, Thiel S. Towards agile product derivaiton in software product line engineering. *Sixteenth European Conference on Software Process Improvement (EuroSPI 2009)*, 2009.
35. Muccini H, van der Hoek A. Towards testing product line architectures. *Electronic Notes of Theoretical Computer Science* 2003; **82**(6).
36. Bosch. *Presentation: Product Derivation—Research Interests within BOSCH*. University of Limerick: Lero, December 2006.
37. Poppendieck M, Poppendieck T. *Implementing Lean Software Development: From Concept to Cash (The Addison-Wesley Signature Series)* (1st edn). Addison-Wesley Professional: Reading MA, 2006; 304.

#### AUTHORS' BIOGRAPHIES



**Pádraig O'Leary** obtained a first class honours BSc degree in Computer Systems from the University of Limerick in 2005. He obtained his PhD in 2010. He worked as a post-doctoral researcher in Lero—the Irish Software Engineering Research Centre, on a software services project. Prior to joining Lero, Pádraig worked as a software developer in both Fineos and State Street Bank. His main research interests are software process improvement and software product lines. Currently, he is a Post-doctoral researcher with the Reuse in Software Engineering (RiSE) group based in the Federal University of Bahia (UFBA).



**Dr Fergal Mc Caffery** is a Lecturer with Dundalk Institute of Technology. He is leader of the Regulated Software Research Group in Dundalk Institute of Technology and a member of Lero. He has been awarded Science Foundation Ireland funding through the Stokes Lectureship and Principal Investigator Programmes to research the area of software process improvement for the medical device domain. Additionally, he has received EU FP7 research funding to improve the effectiveness of embedded software development environments for the medical device industry.



**Steffen Thiel** is a Professor of Software Engineering in the Department of Computer Science at the Furtwangen University of Applied Sciences, Germany. His research interests are in the area of Strategic Software Engineering and include Software Product Lines, Software and Systems Architecture, Software Visualisation, and Software Evolution. Previously, he was a Research Area Leader within Lero—the Irish Software Engineering Research Centre.



**Dr Ita Richardson** is a Senior Lecturer in the Department of Computer Science and Information Systems at the University of Limerick, Ireland, and a Research Area Leader for Process, Practice and Methods within Lero—the Irish Software Engineering Research Centre. Dr Richardson's main research interests are in software quality and software process improvement, focusing particularly on Global Software Development and regulated industry such as Medical and Health Systems. She publishes internationally and supervises full-time and part-time PhD students. She has received funding from sources such as Science Foundation Ireland, Enterprise Ireland and the European Community.